

Cloudflare ASE Take-Home Assessment — Report

Candidate: <your name> · Date: 2026-09 · Repo: github.com/LiCarry/cloudflare

Live: <https://www.clouddemo.cc.cd> (app) · [/secure](https://secure.clouddemo.cc.cd) (Worker) · <https://tunnel.clouddemo.cc.cd> (Tunnel) · Origin: Railway free tier · everything else: Cloudflare Free plan

1. What I built

A product-catalogue web application published entirely through Cloudflare, covering the three parts of the assessment:

- **Part 1 — Application Services:** the origin runs on **Railway**; `www.clouddemo.cc.cd` is proxied through Cloudflare (orange cloud) with **TLS Full (strict)**, protected by the WAF (Cloudflare Managed Ruleset + a SQL-injection custom rule) and a rate-limiting rule on the login endpoint. Direct access to the origin is refused by an application-level check.
- **Part 2 — Zero Trust:** a **Cloudflare Tunnel** (`cloudflared` as a supervised child process on the origin) publishes the app with zero inbound ports; **Cloudflare Access** with a **One-time-PIN** IdP locks `/secure` down to my identity and `@cloudflare.com`.
- **Part 3 — Developer Platform:** a **Worker** (Wrangler-built) serves `/secure` with the authenticated user's identity — "`${EMAIL}` authenticated at `${TIMESTAMP}` from `${COUNTRY}`" — where the country links to `/flags/${COUNTRY}`. Flags are served from a **private R2 bucket** (`/flags/:CC`) and from **D1** (`/flags-d1/:CC`), both via Worker bindings.

```
visitor → Cloudflare edge (WAF · rate limiting · Access SSO)
├─ /search /login ... → proxied origin on Railway (Express)
├─ /secure /flags/*   → Worker → R2 (private) & D1
└─ tunnel.<domain>/*  → Cloudflare Tunnel → cloudflared → localhost:8080
```

2. Implementation, step by step

2.1 Origin on Railway (Part 1.1–1.2)

Express application (`server.js`) deployed from GitHub as a Railway service; Public Networking generated the `*.up.railway.app` domain (Railway injects `PORT=8080`). Routes: `/` (landing + request trace), `/search` (**deliberately SQL-injectable** search over in-memory SQLite), `/login` + `POST /api/login` (always-401 brute-force target), `/secure` (renders Access identity headers), `/healthz` (liveness, exempt from the Cloudflare-only check).

The domain (`clouddemo.cc.cd` , a free delegated subdomain) was added to Cloudflare with its NS records pointed at Cloudflare. Publishing the app on `www` surfaced a real problem: a proxied CNAME at the platform's public domain is unsupported — Railway requires the **Custom Domain** flow (a dedicated CNAME target + a `_railway-verify` **TXT record**), after which the proxy stays orange and Railway issues the certificate.

2.2 TLS — encryption mode recommendation (Part 1.4)

Full (strict). *Flexible* shows the user a padlock while Cloudflare reaches the origin in plaintext — the padlock is cosmetic; *Full* encrypts but accepts any certificate, leaving MITM room; *Full (strict)* encrypts **and validates**. Railway serves a valid certificate for the custom domain, so strict mode needed zero extra configuration. Customers care because compliance frameworks (PCI-DSS, HIPAA) assume the *entire* path is encrypted.

2.3 WAF and the SQL injection demo (Part 1.5)

Enabled the **Cloudflare Managed Ruleset** (Free plan ships the *Cloudflare Free Managed Ruleset* on by default) and added a custom rule for a deterministic demo:

```
http.request.uri.query contains "%20OR%201%3D1"
or http.request.uri.query contains "%20UNION%20SELECT"
or http.request.uri.query contains "sleep("    → Block
```

The interesting part: the ruleset language matches the **raw, still URL-encoded** query string, and `lower() / url.decode()` turned out not to be available in the editor on this plan — so the rule matches the encoded shapes payloads actually travel in (`' OR 1=1 --` arrives as `q=%27%20OR%201%3D1%20--`). Result: both attack payloads return **403** at the edge (Security → Events shows the matched rule), while `?q=hoodie` returns 200. Reached through the tunnel, the same payload executes on the origin and dumps the whole table (a `UNION SELECT` even leaks the users table) — that before/after contrast is the value of managed rulesets: research-grade protection, zero application changes.

2.4 Rate limiting (Part 1.6)

One rule (Free plan): same-IP requests to `/api/login` exceeding **10 per 10 seconds** are **blocked for 1 minute**. Use case: credential stuffing / brute-force protection — the attacker's guess rate drops from thousands per second to ~10 per minute and the traffic never consumes origin capacity. Measured: `401 x9`, then `429s` — nine origin 401s, then Cloudflare 429s (one 401 reappearing when the sliding window expired, a nice detail that shows the mechanism working).

2.5 Preventing Cloudflare bypass (Part 1.7)

`REQUIRE_CLOUDFLARE=true` enables two checks in `lib/cf-only.js`:

1. **Peer-IP check** — walk `X-Forwarded-For` from the *right* (entries proxies append are unforgeable; left entries are client-spoofable) and allow only Cloudflare's published ranges or loopback (the tunnel).
2. **Shared-secret header** — on Railway the first check alone misfires: its edge normalises `X-Forwarded-For` to the *original visitor IP*, so the Cloudflare edge IP never appears and legitimate proxied traffic looks "direct". Fix: a **Transform Rule** force-sets `X-Origin-Secret: <random>` on every request that traverses Cloudflare (Cloudflare overwrites attacker-supplied values, and reserves the `x-cf-` prefix, hence the name), and the origin compares it against `CF_SHARED_SECRET`.

Demo: direct hits on `*.up.railway.app` → **403 "Direct origin access is blocked"**; via `www.clouddemo.cc.cd` → 200. Why it matters: an attacker with the origin address can otherwise skip the

WAF, rate limiting and bot management entirely. Production-stronger options: origin firewall allowlisting Cloudflare IPs, Authenticated Origin Pulls (mTLS), or tunnel-only reachability (what Part 2 achieves).

2.6 Cloudflare Tunnel + SSO + Access policy (Part 2)

A **remotely-managed tunnel** (`origin-tunnel`) whose token is injected as `TUNNEL_TOKEN` ; the app starts `cloudflared` as a supervised child process with automatic restarts, making an **outbound-only** connection (4 QUIC connections registered, logs show HEALTHY). Public hostname `tunnel.clouddemo.cc.cd` → `http://localhost:8080` (the port from the app's own log — not 3000; a mismatch produces 502s).

Identity: **One-time PIN** (email + code — zero external dependencies). Two self-hosted **Access applications** cover `/secure` on both hostnames (tunnel and www), each with one Include policy: **Emails = mine OR Email domain = `cloudflare.com`** . Incognito visitors hit the Access login and never reach the origin; after OTP the origin receives `cf-access-authenticated-user-email` and renders the identity.

2.7 Worker + R2 + D1 (Part 3)

The Worker (`worker/src/index.js`), built and deployed with **Wrangler**, bound to three routes (`/secure*` , `/flags/*` , `/flags-d1/*`):

- `/secure` — extracts the Access JWT (`Cf-Access-Jwt-Assertion`), **verifies the signature with WebCrypto against the team's JWKS** (`/cdn-cgi/access/certs`), checks issuer/audience/expiry, and renders `<EMAIL> authenticated at <TIMESTAMP> from <COUNTRY>` as HTML with the country linking to `/flags/<COUNTRY>` (country from `request.cf.country`).
- `/flags/:CC` — reads `flags/<cc>.svg` from the **private R2 bucket** via the binding, returns `image/svg+xml` . The bucket has no public URL; objects are reachable only through the Worker.
- `/flags-d1/:CC` — parameterised query into the **D1** table returning the same SVG.

Two debugging stories worth telling: (1) this team's Access JWTs are **RS256**, not the ES256 I had assumed — the verifier now supports both, and Workers' WebCrypto additionally requires `{name:"RSASSA-PKCS1-v1_5", hash:"SHA-256"}` stated explicitly or it throws `Missing field "hash" in "algorithm"` . (2) D1 caps single SQL statements at ~100KB and Serbia's coat-of-arms flag alone is 181KB — the seed script inserts oversized rows empty and assembles them with `UPDATE ... content || '...'` appends, chunked and retried; the round-tripped file is byte-identical (181,634 B).

3. Where I see these products being useful

- **Reverse proxy + WAF:** any internet-facing property gains DDoS absorption, TLS, bot management and a continuously updated WAF — ideal in front of legacy apps that cannot be modified quickly.
- **Managed Rulesets:** retail (Magecart/skimming), healthcare and finance (compliance), WordPress/CRM estates — protection without a dev cycle.
- **Rate Limiting:** authentication, checkout and password-reset endpoints; API anti-scraping — cheap capacity insurance.
- **Tunnel:** publishing internal tools (dashboards, admin panels, CI, home labs) with zero inbound ports and no VPN; auditable remote access.

- **Zero Trust Access:** VPN replacement, contractor/partner access to specific apps, M&A scenarios with identities spread across IdPs.
- **Workers:** edge personalisation, A/B routing, API-gateway glue, authorisation enforcement — sub-50ms cold starts, no servers.
- **R2:** zero-egress-fee asset hosting — backups, ML datasets, log archives; the private-bucket + Worker pattern serves paid or entitlement-gated content cleanly.
- **D1:** serverless relational data for read-heavy apps, config/feature-flag stores, session and metadata tables colocated with Workers.

4. How I filled the gaps in my knowledge

- **Platform–Cloudflare interactions:** hit **Error 1000** ("DNS points to prohibited IP") when a proxied CNAME ultimately resolved into Cloudflare's own network, and learned the Custom Domain + TXT-verification flow platforms use to support orange-clouded hostnames.
- **WAF rule semantics:** discovered the ruleset language matches the *raw encoded* URI and that some transformation functions weren't available — rewrote the rule against encoded fragments after the first version silently never matched.
- **Proxy-chain forensics:** Railway normalising `X-Forwarded-For` to the visitor IP broke my "rightmost entry = true peer" check; the shared-secret Transform Rule is the standard remedy and taught me how header injection can carry proof-of-path.
- **Access JWT verification:** assumed ES256; reality was RS256. Learned to read the team JWKS first, support both algorithms, and that Workers' WebCrypto needs the hash spelled out.
- **D1 limits:** `SQLITE_TOOBIG` on the 181KB flag → chunked seeding with UPDATE-concatenation.
- **Tunnels on PaaS:** running `cloudflared` as a supervised child process next to a single-process web service, with the binary downloaded at install time.

5. How would a target customer experience this?

Very positively — time-to-first-value is exceptional. Proxy + TLS went from zero to secure within the hour with no code changes; the tunnel removed a VPN-class problem in minutes; Access delivered SSO with a policy UI instead of weeks of IdP integration. One coherent dashboard (DNS, WAF, rate limiting, Zero Trust, Workers) keeps the mental model simple, and the free tiers are generous enough to prototype everything for real.

Frictions to be honest about: platform-specific quirks cost real debugging time — "orange cloud in front of orange cloud" (Error 1000), header normalisation hiding the Cloudflare edge IP, and a WAF expression language that matches encoded input. Free tiers have limits that surface as puzzling failures (D1's 100KB statement cap, one rate-limiting rule, sleeping free instances with cold starts). None of these are product flaws so much as distributed-systems reality — and every one had a clean fix. The Wrangler workflow with local emulation for R2/D1 was genuinely pleasant: bindings could be developed offline before the first deploy.

6. Access instructions for reviewers

What	URL / command
Application (proxied, WAF + rate limiting)	<code>https://www.clouddemo.cc.cd/</code>
SQLi demo (blocked at edge)	<code>curl -si "https://www.clouddemo.cc.cd/search?q=%27%20OR%201%3D1%20--" → 403</code>
Rate-limit demo	<code>https://www.clouddemo.cc.cd/login</code> — "Fire 20 requests" button
Origin via Tunnel (Zero Trust)	<code>https://tunnel.clouddemo.cc.cd/</code>
Protected path (Worker, HTML identity)	<code>https://www.clouddemo.cc.cd/secure</code> — Access policy: candidate + @cloudflare.com , OTP login
Flags from R2 / D1	<code>https://www.clouddemo.cc.cd/flags/CN</code> · <code>/flags-d1/CN</code> (<code>x-flag-source</code> header distinguishes)
Direct origin (blocked)	<code>curl -si https://<app>.up.railway.app/ → 403</code>

Reviewers with an @cloudflare.com address are covered by the Access email-domain rule on /secure .